



C.V. RAMAN GLOBAL UNIVERSITY
Department of Computer Science Engineering

CASE STUDY REPORT
On
CITY COPORATION AUTOMATION SYSTEM

Academic Year: 2026 - 27

Programme: B.Tech

Course Code: CS387

Title of the Case Study : City Corporation System

Semester & Section: 5TH , Group 8

Course Title : Software Engineering Lab

Objectives:

1. To automate the main services of a city corporation (certificates and licences, property tax and bill payment, and complaint handling) through a single secure web portal, so citizens don't need to visit offices repeatedly.
2. To make the process transparent and easy to monitor, with real-time status tracking and SMS/e-mail alerts for citizens, and dashboards and reports for officers and administrators.

Course Outcomes (CO):

1. Apply software engineering principles to analyse the problem and prepare the requirements. This is shown by the incremental process model, the SRS with functional and non-functional requirements, and the feasibility study.
2. Design, test and manage the software using standard models and techniques. This is shown by the use case, DFD, ER, class and sequence diagrams, the test cases, the COCOMO effort estimate, and the project schedule and risk plan.

Members:

Anubhab Mishra	2401020323
Shatarupa Panda	2401020925
Dinesh Kumar Swain	2401020340
Saket Singh	2401020290
Md Aman Uddin	2401020358

Bloom's Taxonomy Level: L4-L5

Assessment Method: Report with Presentation

Issue Date: 30/09/2026

Submission Date: 07/10/2026

Faculty Name:

Signature:

TABLE OF CONTENTS

1. Introduction
2. Problem Statement and Proposed System
3. Feasibility Study
4. Software Process Model
5. Requirement Analysis
6. System Design
7. Module Description
9. Implementation Details
10. Testing
11. Project Planning and Management
12. Conclusion

1.Introduction

Background

A city corporation (municipal corporation) is the local government body responsible for the everyday civic life of a city. Its duties include issuing birth and death certificates, trade licences and building permissions, collecting property tax and water charges, maintaining roads, drains and street lights, managing solid waste, and responding to the grievances of citizens. In many cities these services are still delivered through paper files, counters and repeated visits to ward offices. A citizen who wants a simple certificate may have to visit several desks, wait in queues, and return again because a document was missing or a file moved slowly from one table to another.

The City Corporation Automation System (CCAS) described in this report is a web-based software system that automates the major citizen-facing and back-office processes of a city corporation. The project has been planned and documented by following standard software engineering practice: a process model was chosen, requirements were specified, the system was modelled with UML, data flow and entity-relationship diagrams, and a testing and project management plan was prepared.

Objectives

- To provide a single online portal through which citizens can apply for certificates and licences, pay taxes and bills, and lodge complaints.
- To reduce the turnaround time of municipal services by moving applications through a defined digital workflow with clear responsibilities.
- To bring transparency by allowing citizens to track every application and complaint in real time and to receive SMS or e-mail alerts.
- To give officers and administrators accurate dashboards and reports for monitoring revenue collection, pending work and complaint resolution time.
- To store all records securely in a central database, with role-based access and an audit trail of important actions.

2. Problem Statement and Proposed System

2.1 Existing System and Its Limitations

In the existing manual or partly computerised arrangement, each department of the corporation maintains its own registers and files. The main limitations are listed below.

- Time and effort: citizens must visit the office physically, often more than once, for submission, verification and collection.

- Lack of transparency: there is no easy way to know the status of an application or a complaint; this encourages unofficial follow-ups.
- Data duplication and loss: paper records can be misplaced or damaged, and the same citizen data is entered again in every department.
- Delayed revenue collection: tax notices are sent late and payments are accepted only at counters during office hours.
- Weak monitoring: managers cannot easily see pending files, overdue complaints or collection figures; reports are prepared manually and are often outdated.

2.2 Problem Statement

There is a need for an integrated, secure and easy-to-use software system that automates the services of a city corporation so that citizens can access them online at any time, officers can process requests through a clear workflow, and administrators can monitor the entire operation through accurate, up-to-date reports.

2.3 Proposed System

The proposed CCAS replaces the manual workflow with a centralised web application. A citizen registers once and then uses the same account for every service. Applications and complaints are routed automatically to the responsible department, and each state change is recorded and notified to the citizen. Payments are made through an online gateway and a receipt is generated instantly. The main advantages of the proposed system are:

- 24x7 availability of services and reduced footfall at municipal offices.
- Faster processing through a defined workflow with time limits (service level targets) for each step.
- A single source of truth for citizen, property, payment and complaint data.
- Complete traceability: every action is time-stamped and linked to a user.
- Better decision making through dashboards and downloadable reports.

3. Feasibility Study

Before starting development, the feasibility of CCAS was examined from four angles.

Technical feasibility. The project uses mature, freely available technologies (HTML5, CSS3, JavaScript, Node.js and MySQL) that are well documented and widely supported. The development team has working knowledge of these tools, and the application can run on a modest cloud server. Hence the system is technically feasible.

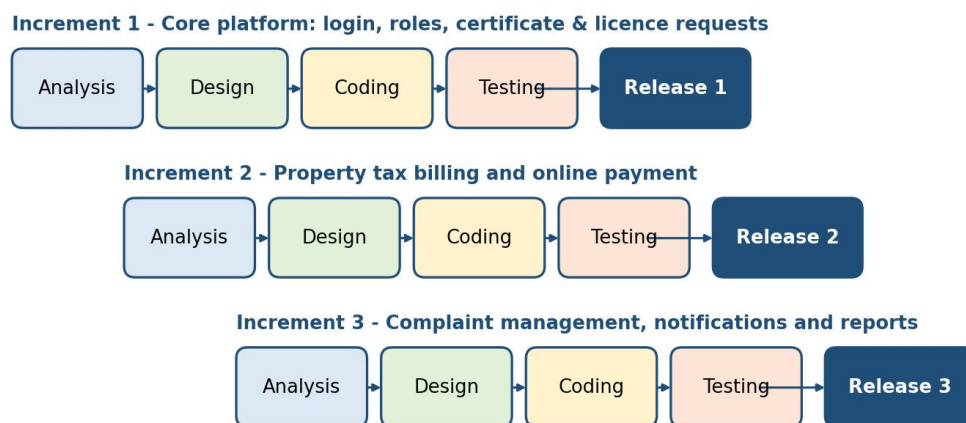
Economic feasibility. The software is built on open-source components, so there are no licence costs. The recurring cost consists of hosting, domain, SMS credits and payment gateway charges, which are small compared with the cost of paper handling, counter staff time and delayed collection that the system reduces. The benefits clearly outweigh the cost.

Operational feasibility. The interface is kept simple and available in a mobile-friendly form so that citizens with limited computer experience can use it. Officers continue to follow their existing approval hierarchy, only in digital form, so resistance to change is expected to be low. Short training sessions and a help section are planned.

Schedule feasibility. The work has been divided into three increments that can be completed in about sixteen weeks (see Section 11). The incremental approach allows a useful version to be delivered early, so the schedule is realistic.

4. Software Process Model

The **Incremental Process Model** was selected for CCAS. In this model the complete requirement set is understood at a high level, but the system is built and delivered in a series of increments. Each increment passes through analysis, design, coding and testing, and adds new working functionality to the product delivered earlier (Figure 1).



This model was preferred over the classical waterfall model for the following reasons:

- Core services such as login and certificate requests have stable requirements and can be delivered first, while less certain features (analytics, notifications) are refined later.
- Users can see and try an early release, and their feedback is incorporated into the next increment.
- Risk is spread over time; defects found in one increment are corrected before the next one begins.
- A small team can work on one increment at a time without needing all the staff at once.

The three increments and their main deliverables are: **Increment 1** - user registration, role-based login, certificate and licence application with document upload and officer approval; **Increment 2** - property register, tax calculation, online payment and receipts; **Increment 3** - complaint management, SMS/e-mail notifications, dashboards and reports.

5. Requirement Analysis

5.1 User Classes

The Software Requirements Specification (SRS) was prepared in the manner suggested by IEEE Std 830. Four classes of users interact with the system, as described in Table 1.

User class	Description and main privileges
Citizen	Registers, applies for services, pays tax and bills, lodges complaints, tracks status, downloads certificates and receipts.
Department Officer	Verifies documents, approves or rejects applications, assigns complaints to field staff, views department reports.
Field Staff	Receives assigned complaints, updates progress and uploads a photo as proof of resolution.
Administrator	Manages users, departments, wards, services and fee structures; views corporation-wide analytics and audit logs.

5.2 Functional Requirements

ID	Requirement
FR1	The system shall allow citizens to register with e-mail or mobile number and verify the account through a one-time password (OTP).
FR2	The system shall authenticate users and give each role access only to its permitted functions.
FR3	Citizens shall be able to apply online for services such as birth/death certificate, trade licence and building plan approval, and upload supporting documents.
FR4	Officers shall be able to verify, approve, reject or return an application with remarks.
FR5	On approval the system shall generate a downloadable certificate (PDF) carrying a QR code for verification.
FR6	The system shall maintain a property register and compute property tax and water charges automatically, including penalty after the due date.

5.3 Non-Functional Requirements

- **Performance:** common pages shall load within 3 seconds and the system shall support at least 500 concurrent users.
- **Security:** passwords are stored as salted hashes, all traffic uses HTTPS, inputs are validated, and access is controlled by roles.
- **Usability:** a clean, responsive interface with simple forms, clear error messages and support for regional languages in future.
- **Reliability and availability:** target availability of 99% with daily database backups and transaction safety for payments.
- **Maintainability and scalability:** modular code with a layered architecture so that new services can be added without rewriting existing modules.

5.4 Hardware and Software Requirements

5.5 Use Case Model

Item	Specification
Client device	Any computer or smartphone with a modern web browser and 2 GB RAM
Server	4 vCPU, 8 GB RAM, 100 GB SSD, Ubuntu Server LTS, Nginx web server
Front end	HTML5, CSS3, JavaScript, Bootstrap
Back end	Node.js with Express (REST API), JSON Web Tokens for sessions
Database	MySQL 8
Tools	Visual Studio Code, Git and GitHub, Postman, draw.io
External services	Payment gateway (sandbox for testing), SMS and e-mail gateway

Figure 2 shows the use case diagram of CCAS. The Citizen interacts with the services on the left of the system boundary, while the Administrator, Department Officer and Field Staff use the management functions on the right. Common functions such as login are shared by all roles.

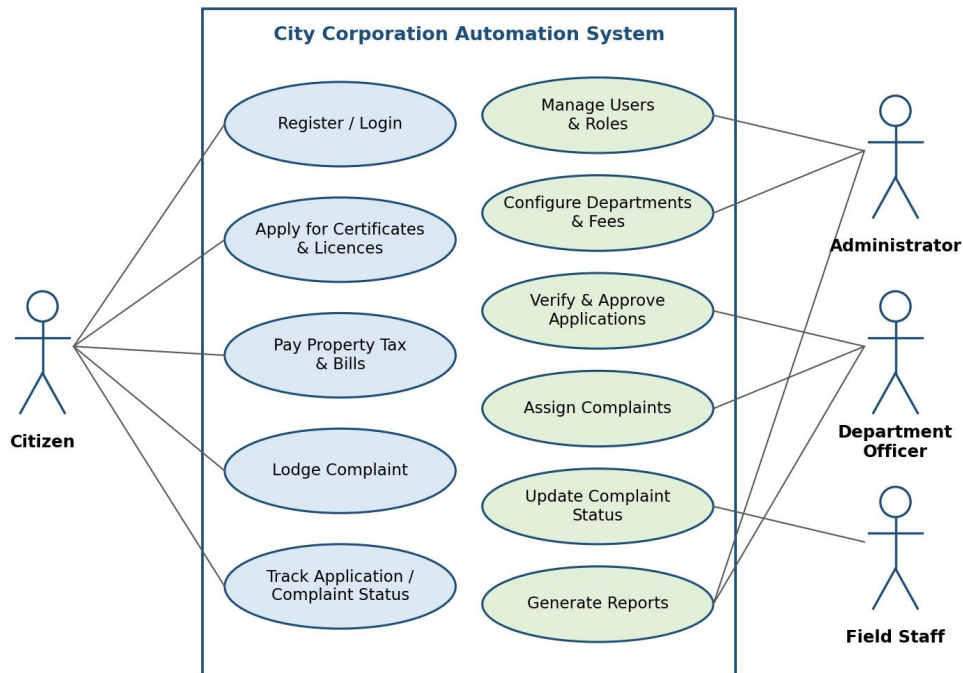


Figure 2

Field	Description
Actor	Citizen (primary), Department Officer, Notification Service
Pre-condition	The citizen is registered and logged in.
Main flow	1. Citizen selects Lodge Complaint. 2. Enters category, description, location and optional photo. 3. System validates the data and saves it with status Open. 4. System generates a complaint ID and sends an acknowledgement. 5. Complaint appears in the queue of the concerned department.
Alternate flow	If a mandatory field is missing or the photo exceeds 5 MB, the system displays an error and returns to the form.
Post-condition	A complaint record exists in the database and can be tracked by the citizen.

6.

System Design

6.1 System Architecture

CCAS follows a three-layer architecture (Figure 3). The **presentation layer** consists of the web pages used by citizens, officers and administrators. The **application layer** exposes REST services that contain the business rules: authentication and access control, certificate and licence handling, tax and payment, complaint management, notification and reporting.

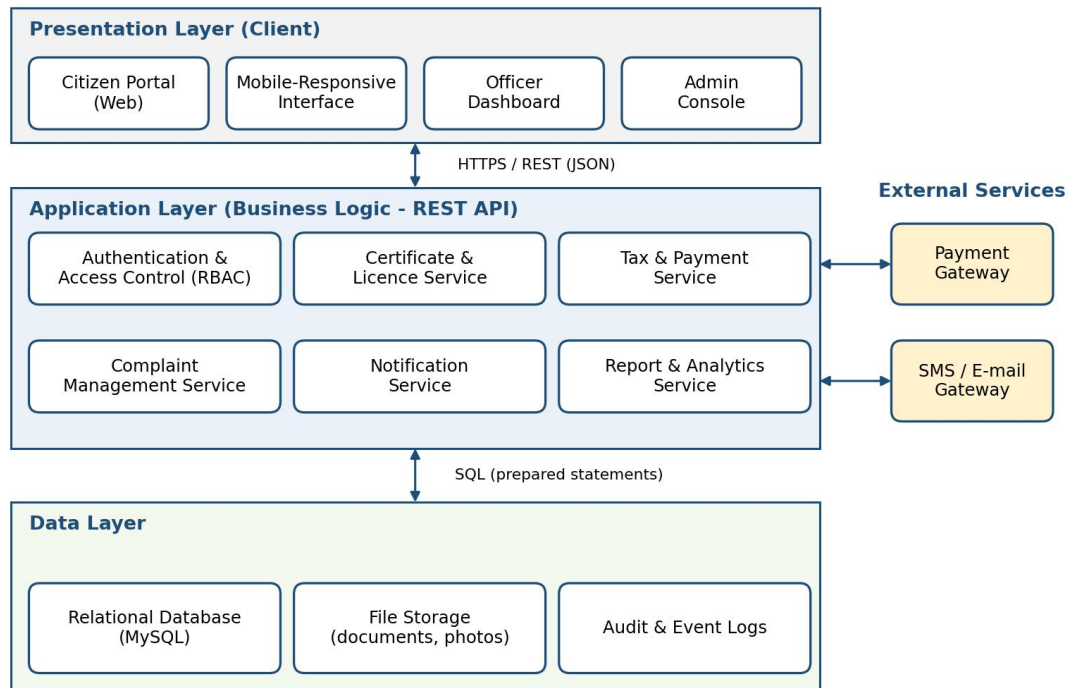


Figure 3

6.2 Data Flow Diagrams

The context-level DFD (Figure 4) shows CCAS as a single process interacting with four external entities. Citizens send applications, complaints and payments and receive certificates, receipts and status updates. Officers send approvals and assignments and receive pending tasks, alerts. The Administrator supplies configuration and receives reports, and the Payment Gateway exchanges payment requests and confirmations with the system.



Figure 4: Level-0 (context) data flow diagram

The level-1 DFD (Figure 5) breaks the system into five main processes: P1 Authentication, P2 Service Requests, P3 Tax and Payment, P4 Complaint Handling and P5 Reports and Analytics.

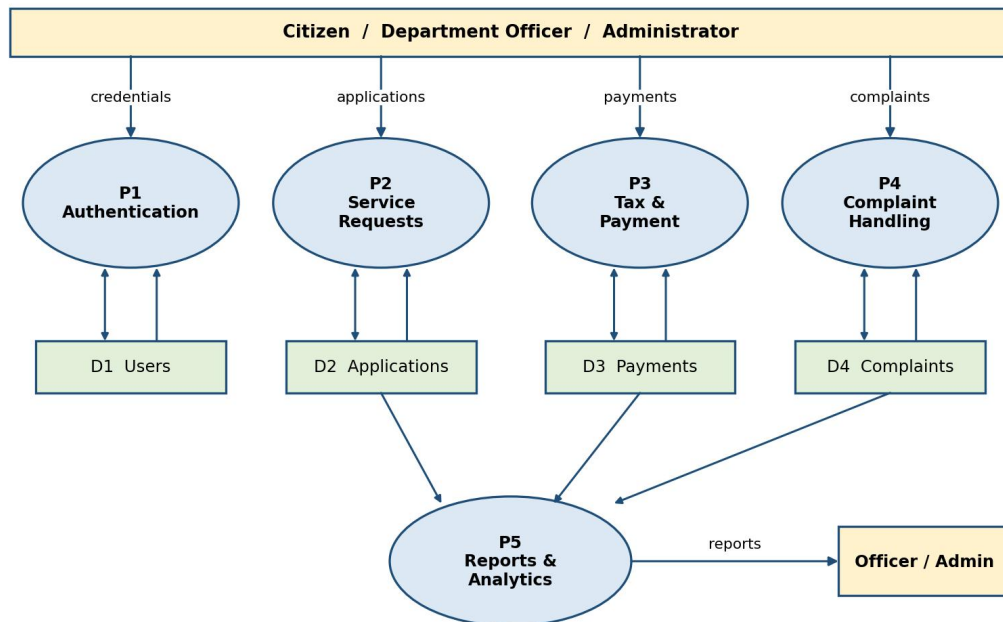


Figure 5: Level-1 data flow diagram

6.3 Database Design

The entity-relationship diagram in Figure 6 shows the main entities and their relationships. One User can submit many Applications, lodge many Complaints, own many Properties and make many Payments. One Department handles many applications and complaints. Each Property has many yearly Tax Bills, and each Tax Bill may be settled by one or more Payments. The design is normalised to the third normal form to avoid redundancy; foreign keys maintain referential integrity.

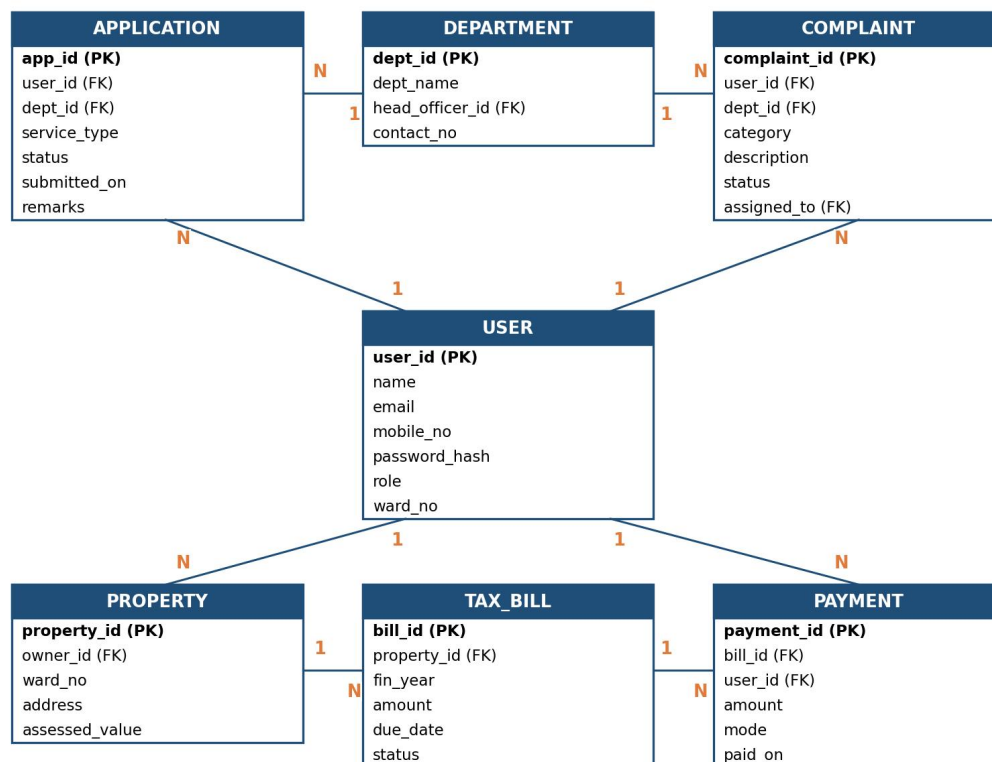


Figure 6: Entity-relationship diagram of the CCAS database

6.4 Class Diagram

The object-oriented design is presented as a UML class diagram in Figure 7. The base class **User** holds the common attributes and the login behaviour. **Citizen**, **Officer** and **Administrator** specialise it with their own attributes and operations (inheritance). Citizens are associated with Payment, Application and Complaint objects; officers approve applications and resolve complaints; administrators manage departments. This structure maps directly to the tables of the database and to the service classes of the application layer.

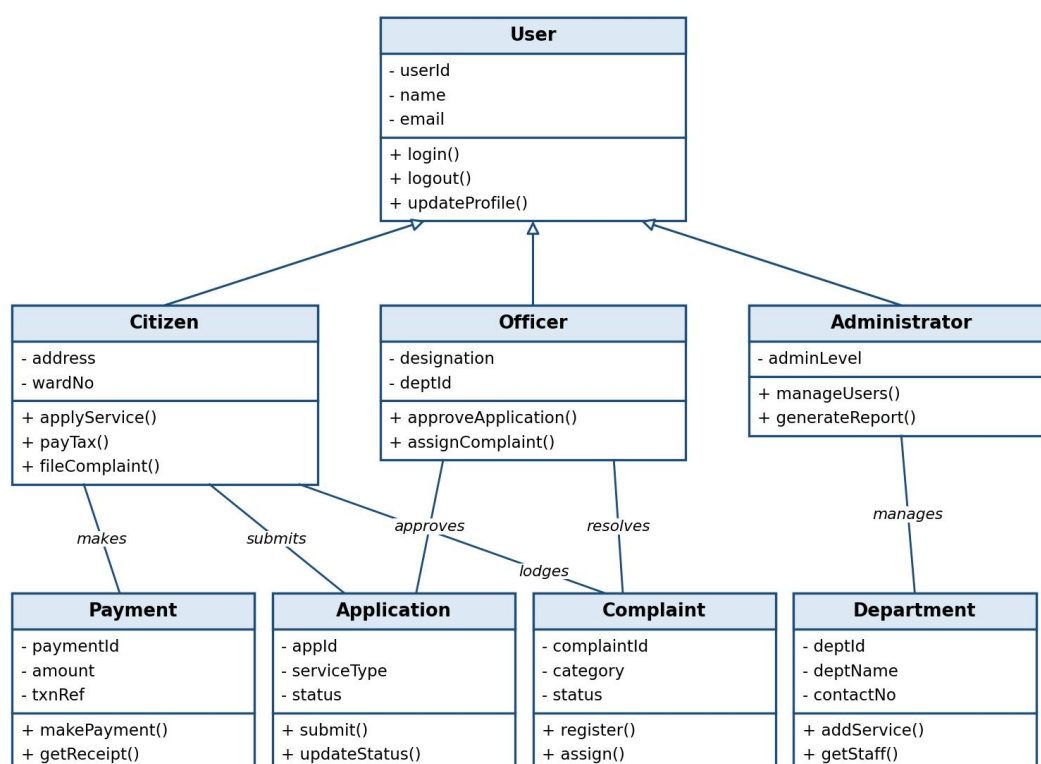


Figure 7: UML class diagram of CCAS

6.5 Sequence Diagram

Figure 8 shows the sequence of interactions for the complaint life cycle. The citizen submits a complaint through the web portal; the complaint service validates it, stores it with status Open and obtains the complaint ID from the database. The notification service sends an acknowledgement by SMS or e-mail. Later the officer opens the complaint, assigns it, and finally marks it as resolved, with each change saved in the database and notified to the citizen.

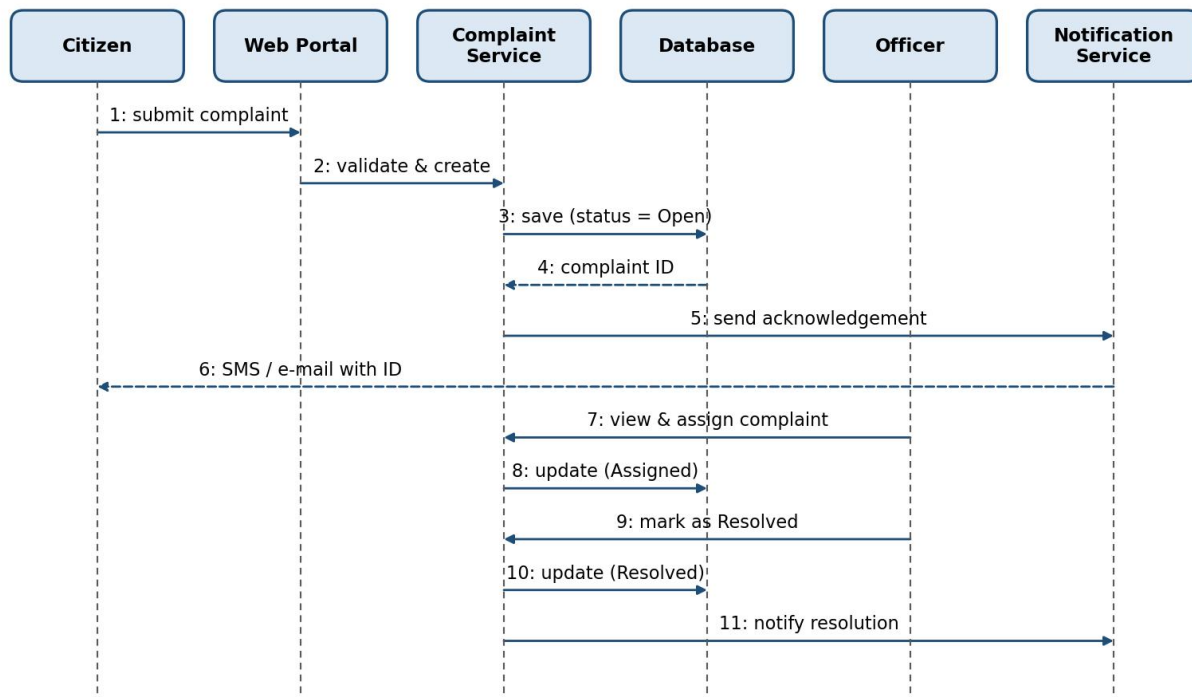


Figure 8: Sequence diagram for the complaint life cycle

7. Module Description

The system is divided into six modules, each corresponding to a service component of the application layer.

Module	Functions
M1: User and Access Management	Registration with OTP verification, login, password reset, role-based access control, profile management, user and role administration.
M2: Certificate and Licence Services	Online application forms, document upload, officer verification and approval workflow, PDF certificate with QR code, application tracking.
M3: Property Tax and Payment	Property register, automatic tax and penalty calculation, bill generation, online payment through gateway, receipt generation and payment history.
M4: Complaint Management	Complaint registration with photo and location, category-based routing, assignment to field staff, status updates and resolution-time tracking.

8. User Interface Design

The user interface was designed with the principles of consistency, simplicity and feedback. The citizen portal uses a left navigation menu, summary cards for the most important information and prominent action buttons for the three most common tasks: applying for a service, paying a bill and lodging a complaint. A "Recent Activity" table shows the latest applications, complaints and payments with their current status. Figure 9 shows the wireframe of the citizen dashboard.

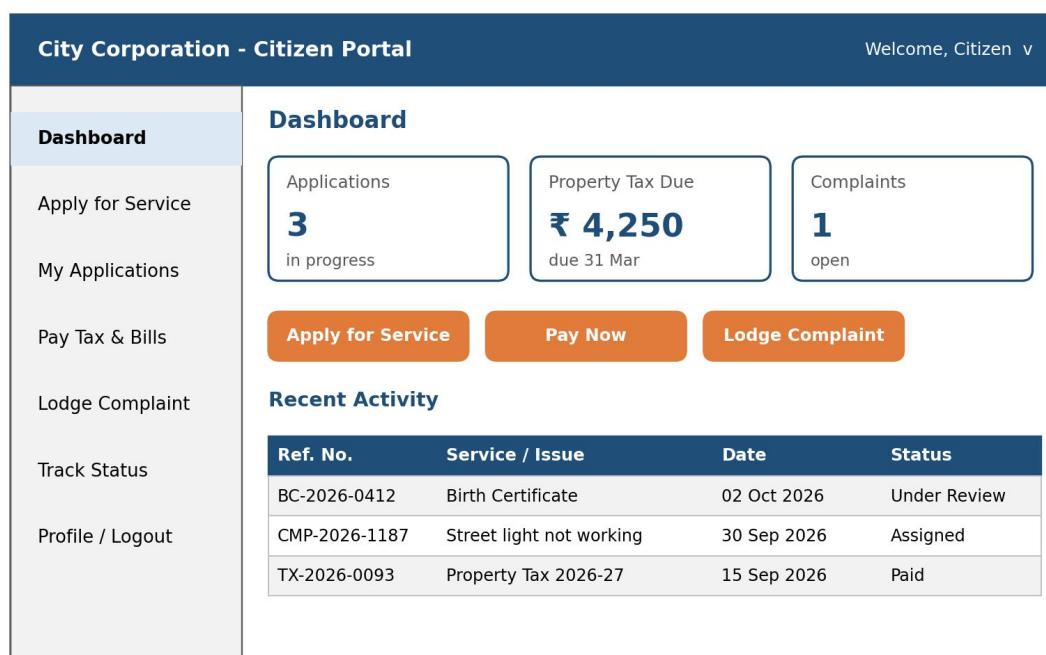


Figure 9: Wireframe of the citizen dashboard

- **Responsive layout:** the pages adapt to phone, tablet and desktop screens.
- **Clear feedback:** every action shows a success or error message; long operations show a progress indicator.
- **Accessibility:** readable fonts, adequate colour contrast and labels on all form fields.
- **Validation:** forms check mandatory fields, formats (mobile number, e-mail) and file types before submission.

9. Implementation Details

9.1 Technology Stack

The front end is written in HTML5, CSS3 and JavaScript with the Bootstrap framework for the responsive layout. The back end is a Node.js application using the Express framework to provide REST APIs that return JSON. MySQL stores all persistent data, and uploaded documents are kept in a protected file store with only the file path saved in the database. Git is used for version control so that every change made by the team members is tracked.

9.2 Security Measures

- Passwords are hashed with a salted algorithm (bcrypt) and are never stored in plain text.
- Sessions are maintained with signed JSON Web Tokens that expire after a fixed time.
- All database queries use prepared statements to prevent SQL injection; user input is escaped to prevent cross-site scripting.
- Role-based access control is enforced on the server for every request, not only in the user interface.

9.3 Coding Standards

The team followed consistent naming conventions, small single-purpose functions, comments for complex logic and a common folder structure separating routes, controllers, services and models. Code was reviewed by a second team member before being merged into the main branch.

10. Testing

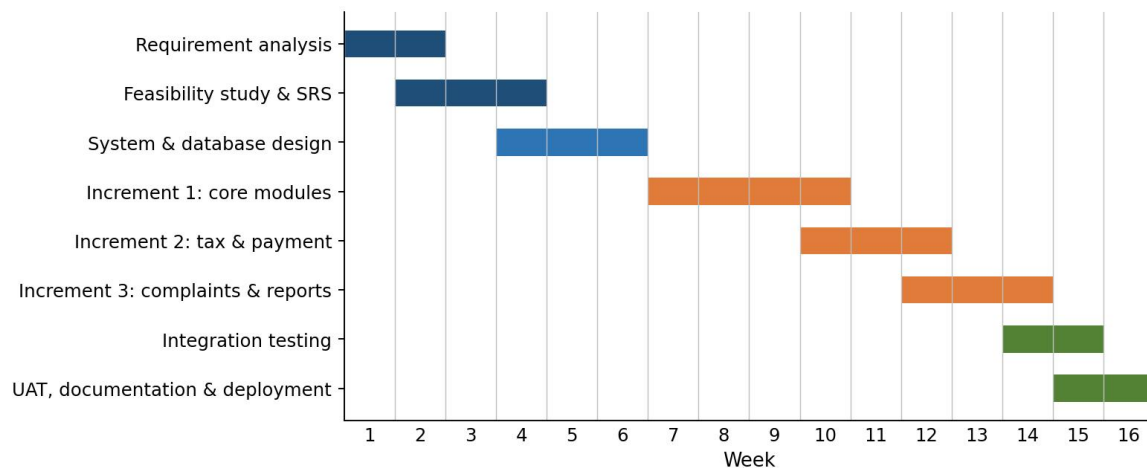
Testing was planned alongside development so that each increment was verified before release. The following levels were used: unit testing of individual functions such as tax calculation and validation; integration testing of modules working together, for example application submission followed by notification; system testing of the complete application against the SRS; and user acceptance testing (UAT) by sample users representing citizens and officers. Black-box techniques such as equivalence partitioning and boundary value analysis were used to design inputs, and white-box testing was applied to critical logic such as penalty calculation and payment status handling.

ID	Module	Test scenario	Expected result
TC01	Login	Login with valid credentials	User reaches the dashboard of his or her role
TC02	Login	Five wrong passwords in a row	Account is temporarily locked and a message is shown
TC03	Registration	Register with an e-mail that already exists	Error: account already registered
TC04	Certificate	Submit a birth certificate request with required documents	Application ID generated, status Submitted
TC05	Certificate	Upload a file larger than 5 MB or of a disallowed type	File rejected with a clear message
TC06	Tax	View bill after the due date	Amount includes the correct penalty

11. Project Planning and Management

11.1 Project Schedule

The project is planned for sixteen weeks. Requirement analysis, the feasibility study and design take the first six weeks; the three increments follow, and the final weeks are used for integration testing, user acceptance testing, documentation and deployment. The Gantt chart in Figure 10 shows the overlap of activities.



11.2 Effort Estimation using Basic COCOMO

The effort required to build the complete system was estimated using the Basic COCOMO model of Boehm. The size of the system is estimated at about 14 KLOC (thousand lines of code). The project is of the organic type because it is a well-understood application developed by a small team, so the constants are $a = 2.4$, $b = 1.05$, $c = 2.5$ and $d = 0.38$.

Quantity	Formula	Result
Effort (E)	$2.4 \times (14)^{1.05}$	38.3 person-months
Development time (D)	$2.5 \times (E)^{0.38}$	10.0 months
Average staff size	E / D	3.8 persons (about 4)

11.3 Risk Management

Risk	Impact	Mitigation
Changes in requirements	Medium	Incremental delivery and early feedback; change requests reviewed before acceptance.
Security breach or data leakage	High	RBAC, hashing, HTTPS, input validation, audit logs and regular backups.
Payment gateway downtime	Medium	Transaction status check and retry; bill is marked paid only after confirmation.

12. Conclusion and Future Scope

The City Corporation Automation System demonstrates how the services of a municipal body can be moved from paper to a secure, transparent and efficient digital platform. Using the incremental model, the project was analysed, specified and designed with standard software engineering artefacts: SRS, use case model, architecture, data flow diagrams, ER and class diagrams, a sequence diagram, a test plan, and a schedule and risk plan. The proposed system reduces the need for citizens to visit offices, speeds up approvals and complaint resolution, improves revenue collection and gives managers reliable information for decisions.

Faculty Assessment and Remarks

Marks Obtained: _____ / _____

Bloom's Taxonomy Level Achieved: L4 / L5 / L6

Faculty Remarks:

Faculty Name & Signature: _____ Date: _____